

Use of Machine Learning in Big Data Analytics for Insider Threat Detection

Michael Mayhew
United States Air Force Research Laboratory
Rome, NY
michael.mayhew.1@us.af.mil

Michael Atighetchi, Aaron Adler
Raytheon BBN Technologies
Cambridge, MA
{matighet, aadler}@bbn.com

Rachel Greenstadt
Drexel University
Philadelphia, PA
greenie@cs.drexel.edu

Abstract — In current enterprise environments, information is becoming more readily accessible across a wide range of interconnected systems. However, trustworthiness of documents and actors is not explicitly measured, leaving actors unaware of how latest security events may have impacted the trustworthiness of the information being used and the actors involved. This leads to situations where information producers give documents to consumers they should not trust and consumers use information from non-reputable documents or producers. The concepts and technologies developed as part of the Behavior-Based Access Control (BBAC) effort strive to overcome these limitations by means of performing accurate calculations of trustworthiness of actors, e.g., behavior and usage patterns, as well as documents, e.g., provenance and workflow data dependencies. BBAC analyses a wide range of observables for mal-behavior, including network connections, HTTP requests, English text exchanges through emails or chat messages, and edit sequences to documents. The current prototype service strategically combines big data batch processing to train classifiers and real-time stream processing to classifier observed behaviors at multiple layers. To scale up to enterprise regimes, BBAC combines clustering analysis with statistical classification in a way that maintains an adjustable number of classifiers.

Keywords: *trust, machine learning, usage patterns, documents, email, chat, TCP, HTTP, support vector machine, insider threat, big data*

I. INTRODUCTION

Basing information release and consumption decisions solely on client credentials and attributes has become insufficient for today's information rich, highly dynamic, and interconnected operational environments. To illustrate the need for automated trust management approaches, consider the specific context of information sharing within the Joint Information Environment (JIE) [1]. While enterprise service capabilities, such as Defense Information Systems Agency's (DISA) Network-Centric Enterprise Services (NCES) [2], have started to make information available across the enterprise, aspects like authority and reputation of information and trustworthiness of actors that either produce or consume information are currently not recorded, standardized, and analyzed. This omission can easily lead to situations in which mission participants start acting on legally accessible but non-authoritative information or start

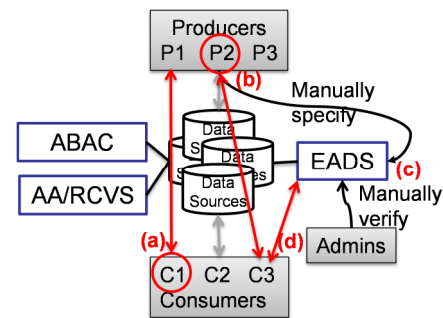


Figure 1. Static Access Control Frameworks

collaborating with authenticated and authorized peers unaware of the fact that their behavior has recently become suspicious.

Meaningful access control decisions on document exchanges must not only specify authorized requests by producers and consumers, but also determine whether the document is an Authoritative Data Source (ADS) or not, based on intrinsic document properties such as a provenance trail.

Figure 1 depicts how existing access control frameworks and document management capabilities regulate information exchanges within Department of Defense (DoD) and Intelligence Community environments. It shows interactions between information producers and consumers over a large set of data sources. NCES provides means for authenticating users via a Robust Certification Validation Service (RCVS), retrieving attributes associated with authenticated users from attribute authorities (AA), and enforcing access control through Attribute Based Access Control (ABAC). NCES is also developing an Enterprise Authoritative Data Sources (EADS) Registry to enable operators to discover authoritative sources to meet their data needs and to enable developers to fuse multiple ADSs into new ADSs which can then be discovered by the operators.

While cyber security monitoring systems have significantly evolved over the last decade, these systems still face a number of limitations.

A. Limitations of Current Analytic Approaches

The main class of currently deployed monitoring solutions focuses heavily on signature-based detection and narrowly target specific parts of the overall systems. Examples of network- and host-based intrusion detection systems include Snort [3] and the Host Based Security System (HBSS) [4]. While these systems provide protection against previously seen attacks, they leave more sophisticated attacks unhandled, including zero-day attacks for which signatures are unknown or targeted

This material is based upon work supported by the Air Force Research Laboratory under Contract No. FA8750-12-C-0011. Distribution A. Approved for public release; distribution unlimited.

attacks that require correlation across systems and layers to be detected, as pointed out in [5].

For tracking user events, a number of commercially available solutions exist in the Security Information and Event Management [6] and cyber monitoring product space today, including ArcSight [7] and the Bit 9 Security Platform [8]. These systems focus on keeping simple statistical summaries of events over time, which are then aggregated and reported along various dimensions. However, they do not take deep content analysis into account, which enables adversaries to utilize exfiltration channels without being detected.

For usage utilization monitoring, a number of solutions exist, including Ganglia [9], Nagios [10], and Zabbix [11]. These systems specialize in performance monitoring and provide operators with dashboard views of the current availability state of the overall network system. While these solutions can readily identify security issues that directly affect performance (e.g., large file transfers), they do not provide any analysis to catch more sophisticated or subtle attacks.

Most current approaches for web filtering, e.g., proxy servers offered by Blue Coat [12], rely on a combination of access policies to restrict the browsable space of the Internet and signatures over known bad content, e.g., JavaScript files. While useful, these proxies typically only deal with privilege escalation/corruption threats and do not provide mitigation of HTTP-based exfiltration channels. Furthermore, they provide little protection against attacks that compromise a frequently visited server and inject new code that shows up only after users have passed authentication.

Finally, big data platforms exist for distributed processing of information. Splunk [13][14] is a well-known instance of a big data processing capability that makes it easy for cyber defenders to establish correlations between disconnected pieces of text information through a specialized query language. However, Splunk requires operators to write explicit rules to classify behaviors into normal vs. abnormal and therefore suffers from the general problem of signature-based approaches mentioned earlier.

B. Limitations of Static Coarse-Grained Access Control Policies

Current access control is based on static policies that tie crypto credentials to attributes that are used by access control rules. Dynamic events, such as subversion of credentials (e.g., theft of Common Access Cards [15]), changes in actor behaviors (e.g., insiders performing illegitimate actions within their privilege realm), and changes in document structures (e.g., edits to Wiki pages), are not addressed at all, leaving systems vulnerable for a considerable period.

In the example context described earlier and shown in Figure 1 (a), consumer C1 starts showing odd behavior, e.g., accessing documents outside of normal operating ranges, yet P1 unknowingly continues to share documents with C1. Similarly, in case (b), P2 suddenly starts producing atypical documents, yet access control schemes do not provide any indication to C3 about P2's abnormal activities.

C. Lack of Automation and Real-time Analytics

While EADS provides basic support for information sharing, it requires manual entry of ADSs (as shown in Figure 1 (c)), which will lead to a sparsely populated database and outdated content. In addition, EADS maintainers are required to verify each newly registered ADS, which is a slow and manual process. The limited number of ADSs for the large amount of documents together with the way documents are used and processed across the enterprise (e.g., recursive mash-ups) also make it difficult for consumers to ascertain trustworthiness of aggregate documents (see Figure 1 (d)).

A similar lack of automation also leads to significant delays in assessment of trustworthiness and detection of actors that have become untrustworthy. Vast amounts of audit data are collected within enterprise environments in the form of server logs which could potentially play a role in access decisions, but this data is typically only used for offline forensics, leading to a situation where "later is too late."

II. RELATED WORK

Analysis of observable behaviors for the purpose of establishing trust models has been a rich research area in cyber security. Chandola et al. present a good survey covering anomaly detection work performed in the past decade [16]. A rich set of literature also exists on spotting behavior differences for the purpose of insider detection [17]. At the network level, a concept called Behavior-Based Network Access Control (BBNAC) performs clustering to identify network behavior profiles [18]. Machines are admitted to the network only if their profiles are deemed normal by their closest cluster of behavior. Compared to BBAC, BBNAC only performs analysis at the network layer while BBAC analyses behaviors at the network, HTTP request, and document layers. Multiple efforts have looked at analysis of HTTP-level behaviors. Examples include the use of machine learning techniques by Ma et al. to predict whether or not HTTP requests are benign [19]. Multiple efforts have investigated the use of behaviors for authentication. Shi et al. use the behavior of cell phone usage to determine whether a cell phone has been lost and thereby changed owner [20]. Similar to BBAC, this work uses machine learning to learn a normal model of usage.

III. BBAC DESIGN

To achieve its functionality, BBAC needs to process vast amounts of data collected by sensors across enterprise environments, including:

- Network flow information, e.g., TCP, and UDP;
- Higher-level transport protocols, e.g., HTTP, XMPP, and SMTP;
- Audit records, e.g., produced by web, OCSP, and DNS servers; and
- Application-level content, e.g., email and chat messages, PDF documents, and wiki pages.

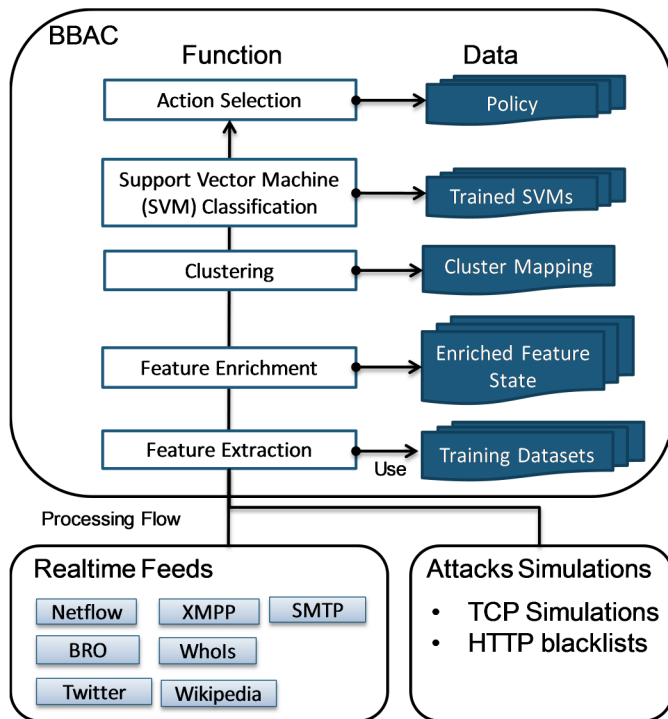


Figure 2. High-level Functional View of BBAC

Figure 2 shows a high-level diagram of the processing flow in BBAC together with the various data sets involved. As shown on the bottom, BBAC ingests a large variety of data from real-time feeds through a feature extraction process. During online use, this data is used for classification purposes. However, for training purposes, BBAC needs to persist and manage training data sets. After parsing the raw observables, BBAC proceeds to a feature enrichment phase, where aggregate statistics are computed and information from multiple feeds is merged into a consistent representation. At this stage, BBAC manages intermediate state required for more complex enrichment functions, e.g., calculating periodicity of events. For unsupervised learning, BBAC uses K-Means++ [21] clustering to group actors, which leads to a mapping function between actors and clusters. Within a cluster, BBAC performs supervised learning through a Support Vector Machine (SVM) [22] that is dedicated per cluster. Finally, action selection determines how to react to classification results obtained from the SVMs. Specific choices such as blocking requests, notifying administrators, or accepting requests are controlled via threshold policies.

A. Scalable Processing Platform

To address scalability concerns related to processing a large number of observables across DoD enterprise systems, BBAC provides a scalable stream processing framework that can take advantage of cloud processing resources. This scalable machine learning framework uses distributed batch processing to cluster and train on archived data and a distributed stream processing-based classifier capable of receiving input from a variety of data sources. In addition to scalability, the framework provides a large degree of flexibility needed to support drastic changes to the prototype throughout the development cycle and customization between target deployments.

In order to achieve the goals of scalability, flexibility, and reliability, BBAC implements its processing framework using Trident [23], a higher level abstraction of the Storm [24] stream processing framework. Storm allows specification of logical topologies of operations on data streams and can transparently distribute stream computation across a large number of compute nodes. Storm also provides reliability through fully fault-tolerant exactly-once messaging semantics, transparently resending information in the case of node failures. Trident adds a higher level of abstraction on top of Storm by providing primitives such as *state*, *query*, *partitions*, and *micro-batches*.

BBAC strategically combines clustering for segmenting the data into groups that have similar behavior with SVMs for building a classifier per cluster for the purpose of detecting behavior changes. Clustering serves several purposes in our framework, most importantly faster training times and more accurate and focused classifiers. These advantages all arise because each classifier is trained for a smaller and more similar group of hosts. In addition to boosting accuracy, clustering also allows us to distribute the training and classification to different cluster nodes. BBAC uses K-Means to form the initial clusters and a prototype implementation of an additional step that uses decision trees.

B. Feature Extraction & Enrichment

The goal of feature extraction is to turn raw observables obtained from sensors (e.g., network packet captures and proxy logs) into a representation that can be used by machine learning algorithms for classification. Feature extraction follows a stream-based paradigm, where new observables continuously become available and the feature extraction logic must keep up with the rate of raw observables entering the system in order to not fall behind in its processing. Feature extraction is inherently IO-bound due to the large amount of data involved and the limited amount of processing required to build up the abstractions useful for machine learning.

The current version of BBAC supports ingestion of a number of different observables, including:

- TCP connection log information from Bro [25], characterizing IP network flows. The features associated with network flows are integer counts associated with various measured attributes of a flow associated with a certain host and time-indexed by day and hour-of-day. The total set of features is broken down into a set of inbound and outbound measurements for each host on an observed network. The features themselves cover aspects such as the total number of connections, the size of traffic sent over the connections, and the average duration of connections.
- HTTP request log information from Bro, characterizing individual HTTP requests and responses. The HTTP features capture observables associated with the request and response headers of HTTP. Note that while BBAC supports generation of features over the payload in general, no payload features have been implemented at this point. Additionally, BBAC can also extract the following features based on WHOIS [26]: queries that are cached in a local database, whether the machine us-

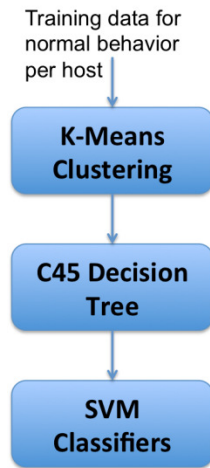


Figure 3. Integration of Decision Trees

es a DHCP IP address (servers will use a static address), and the country for the URL host.

- Stylometric features associated with English written text, e.g., Writeprints [27], extracted from emails, chat messages, or blogs.
- Provenance and edit history information associated with collaboratively developed web pages, in particular pages created via MediaWiki [28]. Features in this category include identity information, document topic, length of change. Also, BBAC computes a knowledge score [29] for each editor, which is an unbounded, positive or negative floating point value that represents the editor's estimated knowledge on the topics of a given edit. The score is equal to the sum of the knowledge scores of all of the editing page's categories for the editing user.

C. Machine Learning Strategy

BBAC employs a strategic combination of supervised and un-supervised machine learning techniques to control scale and classify whether entities, such as hosts, actors, or documents, are showing suspicious behavior. The general assumption is that changes in observable behavior (e.g., changes in outbound traffic patterns) are indicative of attacks executing (e.g., insiders exfiltrating data, bots calling home, or an attack spreading).

The features and methods we use have been used in the anomaly detection research and development community for a significant amount of time and robust implementations are openly available. K-Means++ clustering, Decision Trees, and Support Vector Machine based classifiers are readily available through the Weka [30] machine learning framework.

The BBAC approach is novel in its use of these classifiers with features from a deployed intrusion detection system and on real enterprise data, the use of domain experts to synthesize attack data, and the combination of a clustering pre-processing step to both improve scalability and reduce false positives.

1) K-Means++ Clustering

BBAC strategically combines clustering to segment data into groups that have similar behavior with SVMs to build a clas-

sifier per cluster for detecting behavior changes. For clustering, BBAC uses the K-Means algorithm with K-Means++ initialization as available through the Weka Java API. The goal of the clustering is not to blindly create clusters, but rather to create clusters based on aspects of an entity's behavior. For example, server machines should be in different clusters than desktop machines. An administrative assistant probably accesses different websites at different frequency than a developer who is using cloud compute resources. By placing machines and users into different clusters, we can learn what "normal" is for each one. Our intuition is that these clusters will map to different categories of machines (e.g., servers or desktops, web servers or web crawlers) improving classification rates and resulting in fewer false positives. Multiple classifiers should be able to pick up behavior changes better than a single classifier. For example, an attack vector might communicate using Twitter messages [31]. An infected server that only pulls in a nightly update would be very suspicious if it started using Twitter, however using Twitter is much less suspicious for a desktop machine.

In addition to increasing accuracy through grouping entities with similar behavior into clusters, clustering serves as one of the main means for constraining training times. These advantages arise because each classifier is trained for a smaller and more similar group of hosts. In addition to boosting accuracy, clustering also allows us to distribute the training and classification to different cluster nodes. BBAC uses K-Means to form the initial clusters and a prototype implementation of an additional step that uses decision trees.

2) Support Vector Machines

Unlike clustering, which does not require any training data, Support Vector Machines represent a class of supervised machine learning algorithms that require labeled training data to work. The goal of SVMs is to find an optimal separating hyper plane, given a set of data points that belong to either of two classes. The SVM algorithms are designed to maximize the distance (from the closest points) of either class to the separating plane and at the same time minimize the risk of misclassifying training and test samples.

SVMs have been widely used in the machine learning community and applied to many application domains, including image classification, speech recognition, and protein classification.

One aspect that makes SVMs attractive for use in cyber security is that classification latencies are very low, in the micro-second range on modern computers. Classification performance is made faster by training a classifier, a CPU-intensive process which can take minutes to hours for a dataset of realistic scale and complexity.

BBAC uses SVM classifiers to categorize data as normal versus suspicious. BBAC constructs multiple types of SVM classifiers, e.g., SVM classifiers for TCP connection behaviors, HTTP request behaviors, and Text behaviors, in an attempt to increase accuracy of the detection process.

3) Decision Trees

Trust is an important factor for humans in accepting help from an automated system. A key requirement in trusting a decision-making tool is its ability to provide decision rationale

to human operators. While K-Means clustering followed by a suite of SVM classifiers offers the best balance between efficiency and quality, it is not user friendly. It is not easy for a human operator to understand what the clusters (and their respective classifiers) stand for.

This creates a problem during handling of new hosts for which little observable data exists. Ideally, a system administrator would assign the host to a cluster, and hence traffic for that host could be readily classified using the classifier build for the specific cluster.

To improve the ability to understand the clustering, we extract simple rules that represent the clusters identified in K-Means and then use these rules to assign a host to a group (Figure 3). We represent the rules that define the groups as decision trees and automatically learn the branch conditions (in terms of thresholds on features) under which a host belongs to one cluster vs. another cluster. Decision trees are intuitive and easy for human operators to interpret. Each non-leaf node in a decision tree is a test on an attribute and each branch is a possible outcome of the test. The leaf nodes hold class labels. Using a decision tree is similar to following a control flow diagram.

Building a decision tree that fits the input data optimally is an NP-complete problem. We utilized the C45 algorithm in the Weka Java API to build a decision tree that is mostly consistent with the clustering. The algorithm builds the tree iteratively, starting with the clustering output, adding a node (test attribute) to the tree at each step so that the entropy of the data is reduced.

D. Scalable Training

Training classifiers within BBAC is a batch process that first involves clustering users/computer behaviors using a large set of data samples and then training a SVM for each cluster of samples. Figure 4 shows the training architecture.

Training starts with a user sending an experiment request to a Distributed Remote Procedure Call (DRPC) server. The experiment request contains the parameters of the experiment, including the location of the desired data set. First, the desired training data is read from a disk or database and passed to data modifying functions. Next, the data is passed to a function that splits the input data into multiple combinations of training and testing data in order to perform k-folds cross validation. The number of data folds is specified by the user in the experiment request. Note that in order to train a classifier for deployment BBAC skips the cross validation step, essentially setting the number of data folds to one. After all data folds are created, the training workflow performs clustering on each fold. Next, the workflow trains a classifier for each cluster and then aggregates the resulting classifiers together, creating a meta-classifier for each data fold. The corresponding test data is passed to each meta-classifier resulting in a set of statistics describing the performance of each meta-classifier. The meta-classifiers are (op-

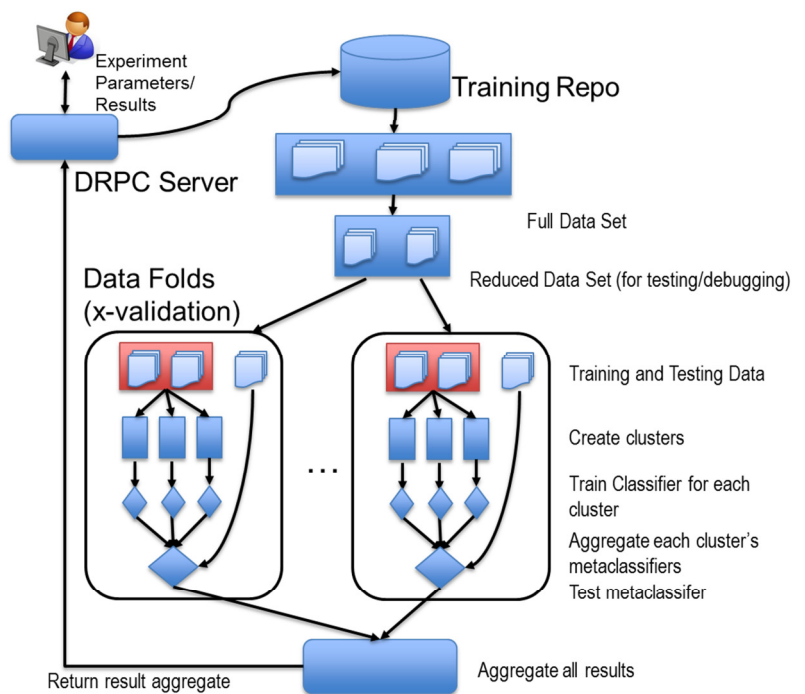


Figure 4. Scalable Training

tionally) saved to a persistent store, test results are aggregated, and performance metrics are returned to the user.

The current parallelization strategy for the training process is based on distributing the following:

- Building of classifiers per cluster (tens of clusters),
- Changing SVM parameter settings (hundreds of settings), and
- Performing k-fold cross validation experiments for each setting and cluster (with k=10 by default).

Overall, this approach allows us to train tens of thousands of classifiers in parallel, leading to a significant speedup over sequential training, even with treating the training of a specific classifier instance as an atomic operation.

E. Scalable Classification

In order to effectively analyze behaviors in DoD enterprise networks, BBAC's classification engine must be able to scale with increasing levels of throughput while maintaining minimal latency. Figure 5 illustrates a target application where an HTTP proxy passes HTTP requests to BBAC for classification and blocks any request tagged suspicious. For this scenario to be realized, BBAC must have latency on the order of tens of milliseconds, a target achieved by the current framework implementation.

The first step in this topology is to parse the input data and extract relevant features. For HTTP requests this involves extracting details such as target URL, time of day, and requester host ID.

The next step takes these features and enriches them using external context. Currently, BBAC issues queries against a

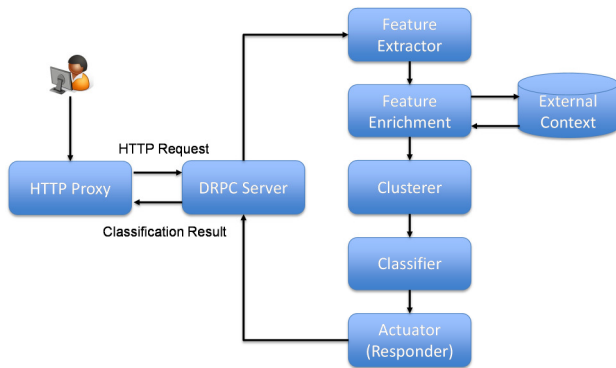


Figure 5. Classification of HTTP Requests

cached WHOIS database and identifies the country that corresponds to a provided IP address. Future examples include integration with DNS queries and host subnet information.

The next step involves aggregating samples over a time interval, summarizing certain features per time interval. This step is not currently used for classifying single HTTP requests, as it would likely introduce a level of latency that would make the concept of BBAC working in line with a HTTP proxy infeasible. For other data types, such as TCP connection headers, this aggregation step allows us to analyze how many connections a computer has opened over a period of time. Consider the example of a server, which normally makes only a few connections to the outside world per hour and is suddenly making thousands of connections per hour. Looking at any one connection request may not reveal suspicious activity. However aggregating connection requests over a given time period may reveal the suspicious behavior. A database is used to aggregate requests, and currently aggregates data into hourly summaries. Which time periods are appropriate for the detection of varying attacks is an area for future research.

BBAC's modular approach to feature extraction, enrichment, and aggregation enables flexibility to experiment with a wide range and types of features during prototype development. This enables future extensions, for instance, to investigate a wide range of enrichment strategies for more accurate classification.

After the feature enrichment operations are performed, each request/sample is assigned to the correct cluster based on the machine/user ID of the host and passed on to the classifier corresponding to that cluster. Currently these classifiers are Weka-based SVMs. The output of the classification process is passed to at least one of several actuators. The current set of implemented actuators include logging results and updating a system administrator interface. Within the context of the HTTP Proxy example, this entails returning whether or not the request should be released.

F. Elastic Processing Framework

Our framework consists of Storm [24], Zookeeper [32], Accumulo [33], and Hadoop [34] in a set of Ubuntu-based Virtual Machines (VMs). Storm allows us to deploy topologies that are distributed to a set of worker nodes. This allows work, either building a classifier for each cluster or classification of data, to be distributed and accomplished more quickly. Storm

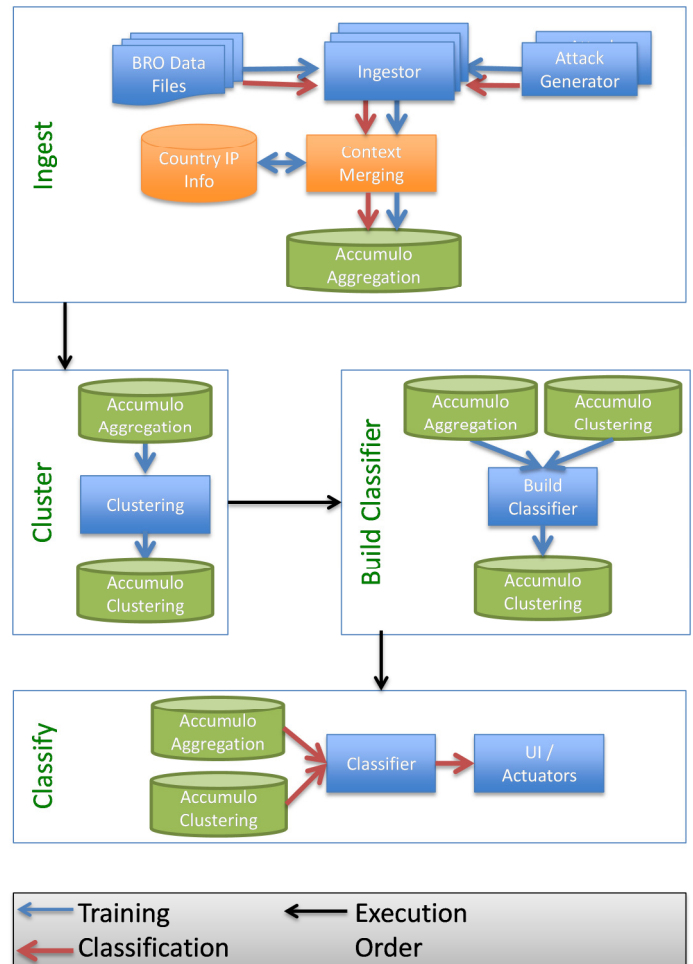


Figure 6. Using Accumulo to Separate System Functions.

uses Zookeeper to manage cluster coordination, allowing Storm to scale to much larger cluster sizes. Accumulo is used as a distributed database to store classifiers, cluster information, and to merge TCP data into aggregated statistics that can be classified. Apache Accumulo uses Hadoop, Zookeeper, and Thrift [35].

The framework allows us to tie different functions together through the Accumulo database. The ingestion, clustering, classifier training, and classification are run as separate Storm topologies that all use the Accumulo database, as shown in Figure 6.

As the load on the system changes, more VMs can be brought online (or taken offline). This requires configuration changes to Accumulo and Storm. In particular, for classifying incoming data, we can simply bring more nodes online that can load the classifier for each cluster and then classify the incoming data. We performed load testing on the system that demonstrated that BBAC processing scales proportionally with the number of processing nodes.

G. Twitter and Email

Authorship verification is intended to test whether a given document is consistent with an author's past writings. Utilizing the feature extraction capabilities of the JStylo framework [36],

we measure feature vectors of various traits of an author's writing style. A centroid is computed as well as the average variance the author has from that centroid. If the test document is farther from the centroid than is usual for the author, it is rejected. If it is nearer, then it is verified. We used two separate data exfiltration schemes for these experiments. The first sends executable data as plain text. The second swaps out every n th word of a base document where each word represents a character. The base document is a past email of the author.

Authorship classification tests whether a given text is written by a specific author from a set of potential authors. This method, known as stylometry, extracts a variety of features from the training and testing texts and uses SVMs for classification. For this task, we also used JStylo, which assists in feature extraction and utilizes Weka for classification. Summarized results from our testing are shown in Table 1.

H. Action Selection

Through the concept of action selection, BBAC enables execution of mitigations in response to observed suspicious events. BBAC provides the following two means for affecting change:

- **Administrator Initiated:** BBAC provides an HTML UI that provides management controls for the system and also displays suspicious events found by the classifiers. This enables administrators to initiate appropriate mitigations based on suspicious events listed in the UI.
- **Risk Adaptive Access Control:** BBAC provides an API for consuming threat status messages describing suspicious events via well-defined XML documents. This enables other components, such as the resiliency controllers being built under the Autonomic Resilient CDS (ARC) effort, to treat BBAC information as sensor input as part of a larger adaptation strategy.

IV. EXPERIMENTAL RESULTS

We evaluated the BBAC prototype capability through a number of internal experiments to determine accuracy, precision, and scalability. Table 1 shows the accuracy metrics conducted via multiple experiments conducted and reported on in previous publications [29] [37]. True Positive (TP) rate is the percentage of attacks correctly identified while False Positive (FP) rate is the percentage of traffic incorrectly classified as an attack. Positive Predictive Value is calculated as $\# TP / (\# TP + \# FP)$. Table 2 shows the BBAC performance metrics.

To evaluate scalability of the compute framework, we executed multiple experiments varying offered load. Figure 7 shows the training time per classifier in relation to the number of classifiers. The plot shows that the training time per classifier is invariant of the total number of classifiers to train. In other words, training one classifier takes about the same as training 160 classifiers due to the use of parallel processing.

Our HTTP experimental results provide both an example of the performance we achieved and also some of the potential operational pitfalls. We envision the tool that results from this work operating in a proxy between the user and the Internet. Depending on operating conditions, it could be more critical to allow all traffic unless obviously bad (very low FP rate), or be

Table 1. Accuracy Metrics.

Data Type	Measured	Result
HTTP	True Positive Rate	99.6%
	False Positive	0.9%
	Positive Predictive Value	99.1%
	Classification latency (locally)	$\mu=19\text{ms}$, $\sigma=5.4\text{ms}$
	Classification latency (Storm)	$\mu=10\text{ms}$
TCP	True Positive Rate	92.9%
	False Positive	0.5%
	Positive Predictive Value	94.2%
	Classification latency (locally)	$\mu=0.74\text{ms}$, $\sigma=0.52\text{ms}$
	Classification latency (Storm)	$\mu=0.02\text{ms}$
Wiki	Correctness of attribution (CA) TP	76%
	Correctness of attribution (CA) FP	1%
	Positive Predictive Value	99%
	Classification latency (locally)	$\mu=1.3\mu\text{s}$, $\sigma=30.2\mu\text{s}$
Twitter	True Positive Rate	96%
	False Positive Rate	0.18%
	Positive Predictive Value	99.8%
Email	True Positive Rate	93%
	False Positive Rate	0.4%
	Positive Predictive Value	99.99%

Table 2. Performance Metrics.

Metric	Measured	Result
Scalability	Training data set size / Number p of parallel processors	$\sim$ constant, $p=40$
	Training data set size / Training time	$\sim$ constant
Security	Coverage over OWASP Top 10 threat categories	100%
	Adherence to applicable NIST-800-53 security controls (over 150)	100%, 68 controls directly impacted by BBAC
Flexibility	Number of supported document and service types	5, TCP, HTTP, Wikipedia, Email, Chat

conservative and block all traffic that was not clearly safe (very high TP rate). As such, trading off TP and FP rates in different situations is critical in this domain. In a typical week 17 million URLs are accessed at BBN. Even a 0.1% error rate that leads to manual review would result in over 2000 URLs per day (1.7 per minute), a rate that quickly becomes infeasible.

We tested many HTTP settings and features, and achieved the best result by using an individual word vector for each string feature and including the WHOIS data features. Our classifier was a Sequential Minimal Optimization (SMO) classifier with a linear poly-kernel (exponent of 1) and a C of 1. We identified malicious URLs successfully with a rate of 99.6% with good URLs incorrectly classified as malicious at a rate of 1.0%. The mean classification time for a given URL is 45 ms with a standard deviation of 5 ms. Classification at this speed would be acceptable for an inline, real-time system.

V. CONCLUSION AND NEXT STEPS

BBAC provides a cyber analytics capability that is urgently needed to defend DoD networks, namely the ability to assess trustworthiness of actors, services, and documents in real-time, at enterprise scale, and with actionable accuracy.

The concepts and approaches described in this document consist of (1) unsupervised machine learning in the form of K-Means++ clustering to control scalability, (2) supervised machine learning in the form of SVM classifiers to predict whether network flows, HTTP requests, text segments, and document provenance edits are suspicious or not, and (3) a scalable com-

Mean Training Time per Classifier vs Total Number of Classifiers

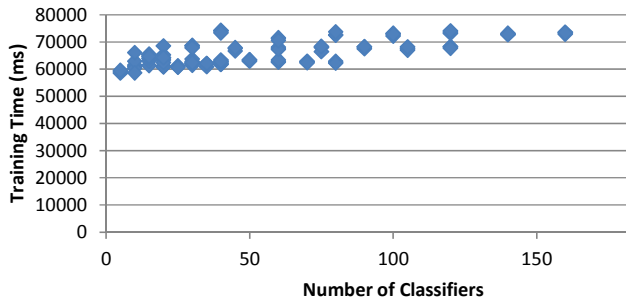


Figure 7. Scaling of Training with the Number of Classifiers.

pute platform that enables the processing to scale with increase load by dynamically adding compute nodes.

Recommendations for future work span both technology transition and research and development. Future work includes deployment and testing at BBN and Drexel University. BBAC can help BBN perform cyber monitoring and incidence response of its intranet. In a university context we expect account compromise to be much more prevalent as username and passwords are frequently shared between students for remote access, whereas BBN employs hardware tokens that are more difficult to share. Furthermore, we expect behavior patterns observed at a University to be less homogeneous when compared to BBN. Both these differences would make installation of BBAC at two separate locations an interesting proposition.

ACKNOWLEDGEMENTS

The authors wish to acknowledge the contributions of the following people who worked on the BBAC project: Jonathan Webb, Fusun Yaman, Jeffrey Cleveland, Rebekah Overdorf, Travis Dutko, Pavan Kantharaju, Joe Muoio, and Ariel Stolerman,

REFERENCES

- [1] M. Dempsey, "Joint Information Environment White Paper," 2013.
- [2] DISA, "DISA Net-Centric Enterprise Services Program Achieves Full Deployment and Full Operational Capability." [Online]. Available: <https://www.disa.mil/News/PressResources/2010/DISA-NCES-Program-Achieves-Full-Deployment>.
- [3] M. Roesch and others, "Snort-lightweight intrusion detection for networks," in *Proceedings of the 13th USENIX Conference on System Administration*, 1999, pp. 229–238.
- [4] DISA, "The Host Based Security System," 2012. [Online]. Available: <http://www.disa.mil/Services/Information-Assurance/HBS/HBSS>.
- [5] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Security and Privacy (SP), 2010 IEEE Symposium on*, 2010, pp. 305–316.
- [6] D. Miller and B. Pearson, *Security information and event management (SIEM) implementation*. McGraw-Hill, 2011.
- [7] ArcSight, "Enterprise Security Manager," *Arcsight ESM*. [Online]. Available: <http://www.arcsight.com/products/products-esm>.
- [8] Bit9, "Bit9 Security Platform." [Online]. Available: <https://www.bit9.com/solutions/security-platform/>.
- [9] M. L. Massie, B. N. Chun, and D. E. Culler, "The ganglia distributed monitoring system: design, implementation, and experience," *Parallel Comput.*, vol. 30, no. 7, pp. 817–840, 2004.
- [10] W. Barth, *Nagios: System-und Netzwerk-Monitoring*. No Starch Press, 2008.
- [11] R. Olups, *Zabbix 1.8 network monitoring*. Packt Publishing, 2010.
- [12] "Blue Coat Systems." [Online]. Available: <http://www.bluecoat.com/>.
- [13] J. Stearley, S. Corwell, and K. Lord, "Bridging the gaps: joining information sources with Splunk," in *Proceedings of the 2010 workshop on Managing systems via log analysis and machine learning techniques*, 2010.
- [14] Splunk.com, "Splunk for Security – Supporting a Big Data Approach for Security Intelligence," 2014. [Online]. Available: http://www.splunk.com/web_assets/pdfs/secure/Splunk_for_Security.pdf.
- [15] DoD, "The Common Access Card," 2012. [Online]. Available: <http://www.cac.mil>.
- [16] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput Surv*, vol. 41, no. 3, pp. 15:1–15:58, Jul. 2009.
- [17] M. B. Salem, S. Hershkop, and S. J. Stolfo, "A survey of insider attack detection research," *Insid. Attack Cyber Secur.*, pp. 69–90, 2008.
- [18] V. Frias-Martinez, J. Sherrick, S. J. Stolfo, and A. D. Keromytis, "A Network Access Control Mechanism Based on Behavior Profiles," in *Proceedings of the 2009 Annual Computer Security Applications Conference*, Washington, DC, USA, 2009, pp. 3–12.
- [19] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, "Learning to detect malicious URLs," *ACM Trans Intell Syst Technol*, vol. 2, no. 3, pp. 30:1–30:24, May 2011.
- [20] E. Shi, Y. Niu, M. Jakobsson, and R. Chow, "Implicit authentication through learning user behavior," in *Proceedings of the 13th international conference on Information security*, Berlin, Heidelberg, 2011, pp. 99–113.
- [21] D. Arthur and S. Vassilvitskii, "k-means++: The advantages of careful seeding," in *Proceedings of the eighteenth annual ACM-SIAM symposium on Discrete algorithms*, 2007, pp. 1027–1035.
- [22] L. Wang, *Support Vector Machines: theory and applications*, vol. 177. Springer, 2005.
- [23] I. Brigadir, D. Greene, P. Cunningham, and G. Sheridan, "Real time event monitoring with trident," in *Georg Kreml et al.(eds.) Real-World Challenges for Data Stream Mining: Proceedings of the 1st International Workshop on Real-World Challenges for Data Stream Mining, RealStream 2013, Prague, Czech Republic, September 27, 2013*, 2013.
- [24] Storm, "Storm home page," Mar-2014. [Online]. Available: <https://storm.incubator.apache.org/>.
- [25] V. Paxson, "Bro: a system for detecting network intruders in real-time," in *Proceedings of the 7th conference on USENIX Security Symposium - Volume 7*, Berkeley, CA, USA, 1998, pp. 3–3.
- [26] L. Daigle, *WHOIS protocol specification*. 2004.
- [27] A. Abbasi and H. Chen, "Writeprints: A stylometric approach to identity-level identification and similarity detection in cyberspace," *ACM Trans. Inf. Syst. TOIS*, vol. 26, no. 2, p. 7, 2008.
- [28] D. J. Barrett, *MediaWiki*. O'Reilly Media, Inc., 2008.
- [29] J. Segall and R. Greenstadt, "The illiterate editor: metadata-driven revert detection in Wikipedia," in *Proceedings of the 9th International Symposium on Open Collaboration*, 2013, p. 11.
- [30] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, and I. H. Witten, "The WEKA data mining software: an update," *ACM SIGKDD Explor. Newsl.*, vol. 11, no. 1, pp. 10–18, 2009.
- [31] M. Farnum, *Bot herders using Twitter for command and control*. 2009.
- [32] "Zookeeper Homepage." [Online]. Available: <http://zookeeper.apache.org/>.
- [33] "Accumulo Project Homepage." [Online]. Available: <https://accumulo.apache.org>.
- [34] "Hadoop Project Page." [Online]. Available: <http://hadoop.apache.org/>.
- [35] "Thrift Homepage." [Online]. Available: <https://thrift.apache.org/>.
- [36] A. W. McDonald, S. Afroz, A. Caliskan, A. Stolerman, and R. Greenstadt, "Use fewer instances of the letter 'i': Toward writing style anonymization," in *Privacy Enhancing Technologies*, 2012.
- [37] Aaron Adler, Michael Jay Mayhew, Jeffrey Cleveland, Michael Atighetchi, and Rachel Greenstadt, "Using Machine Learning for Behavior-Based Access Control: Scalable Anomaly Detection on TCP Connections and HTTP Requests," presented at the 32rd Military Communications Conference (MILCOM 2013), San Diego, CA, 2013.